Decision Trees:

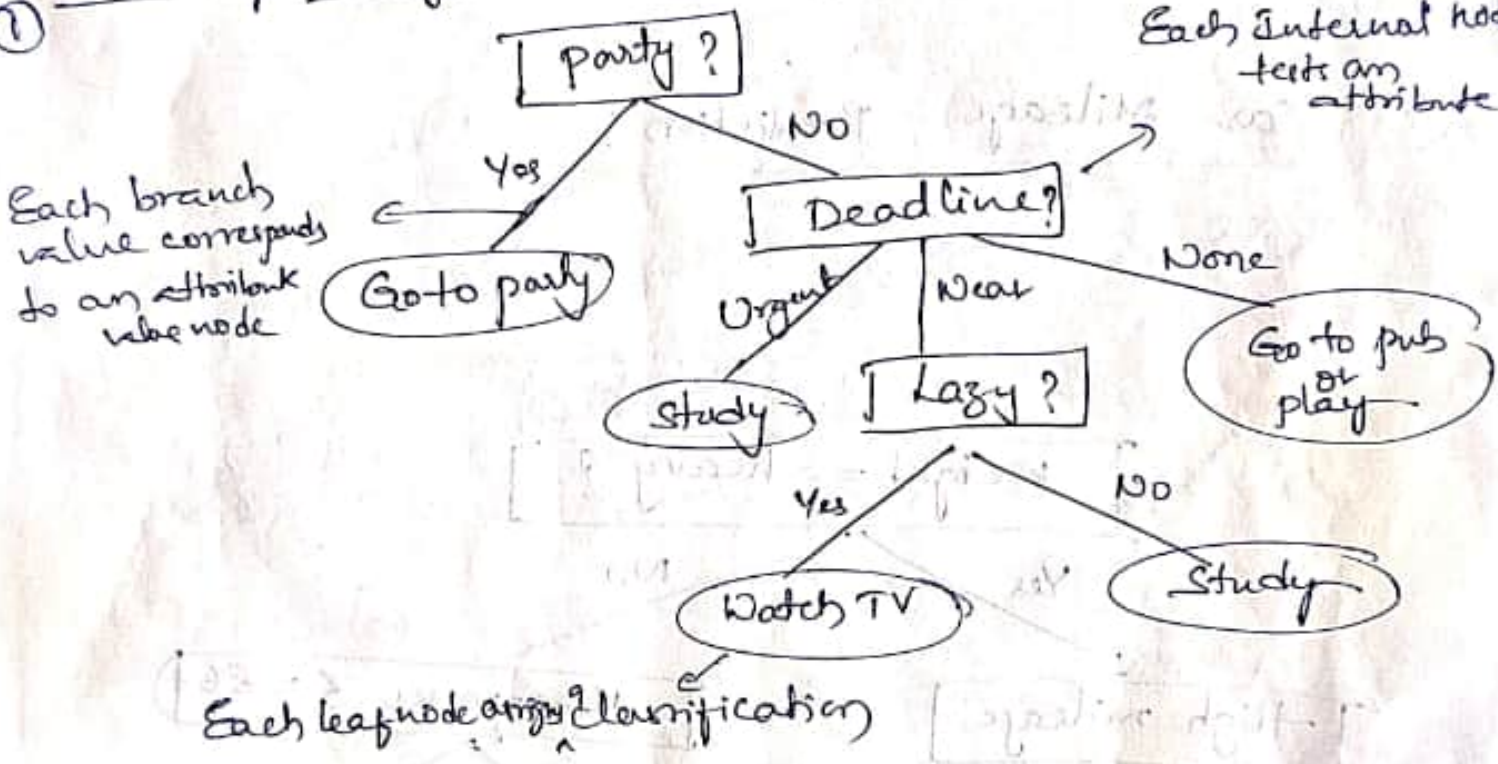
Decision tree is a tree structured classifier.

It consists of decision nodes and leaf nodes.

- Nodes where tests are conducted on the values of the attributes → decision nodes.
- Nodes where the tree stops → leaf nodes

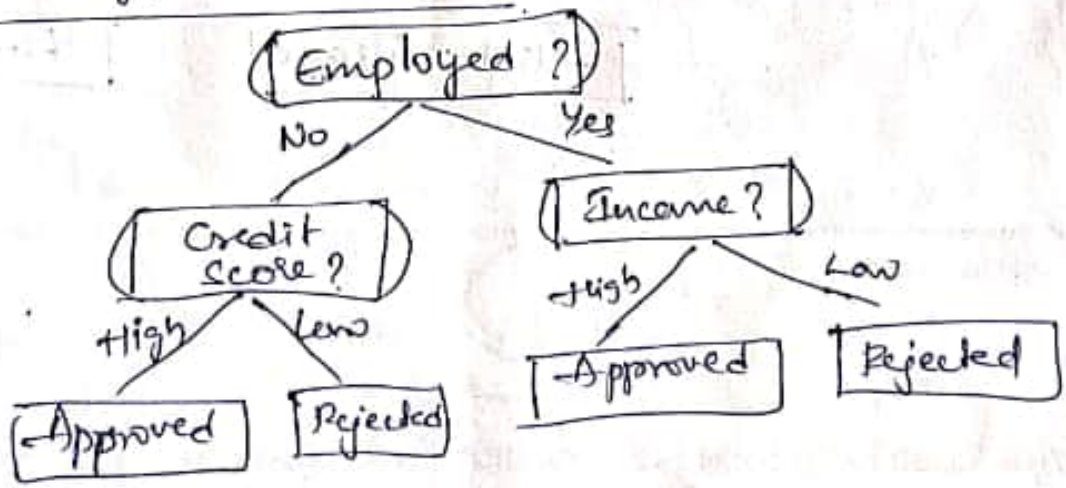
Ex:- Spending the evening

①

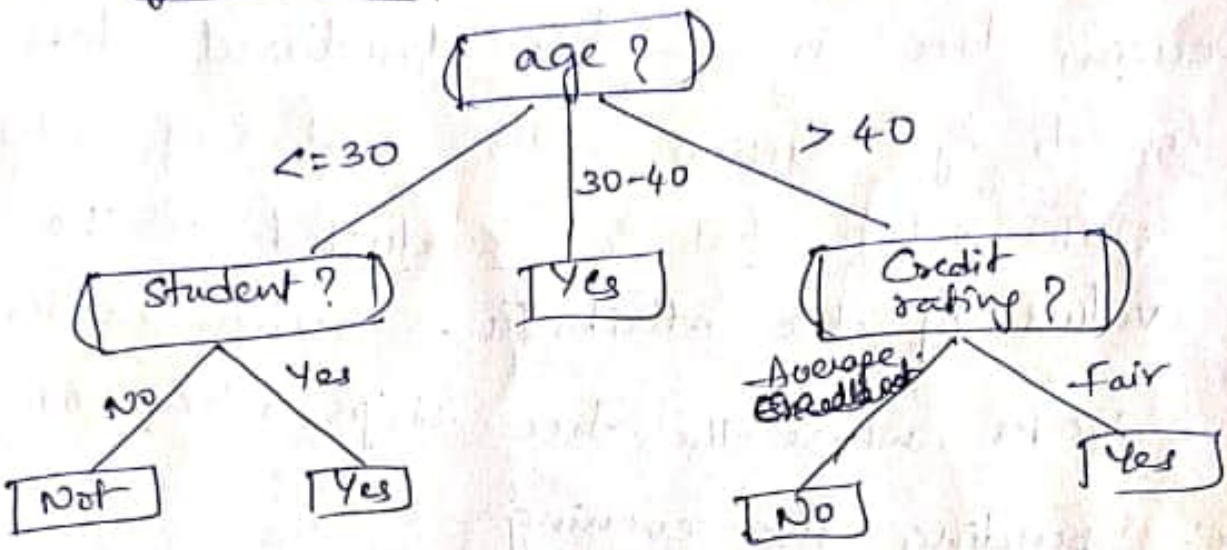


Each leaf node assigns a classification

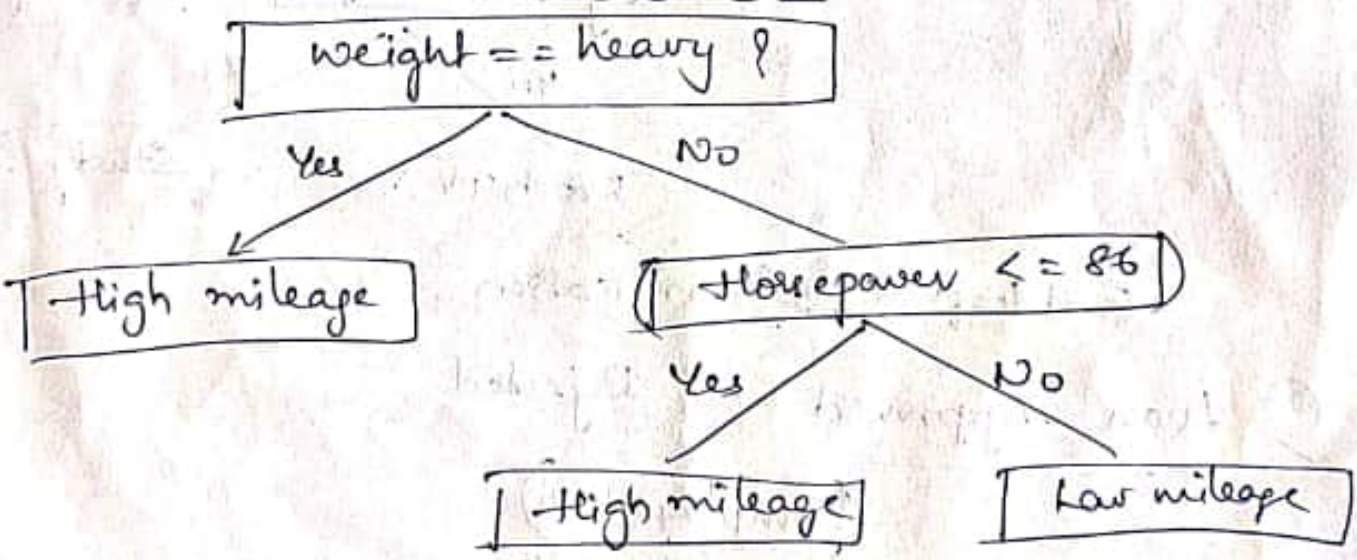
② Loan Approved or Rejected



(3) Buy a computer:

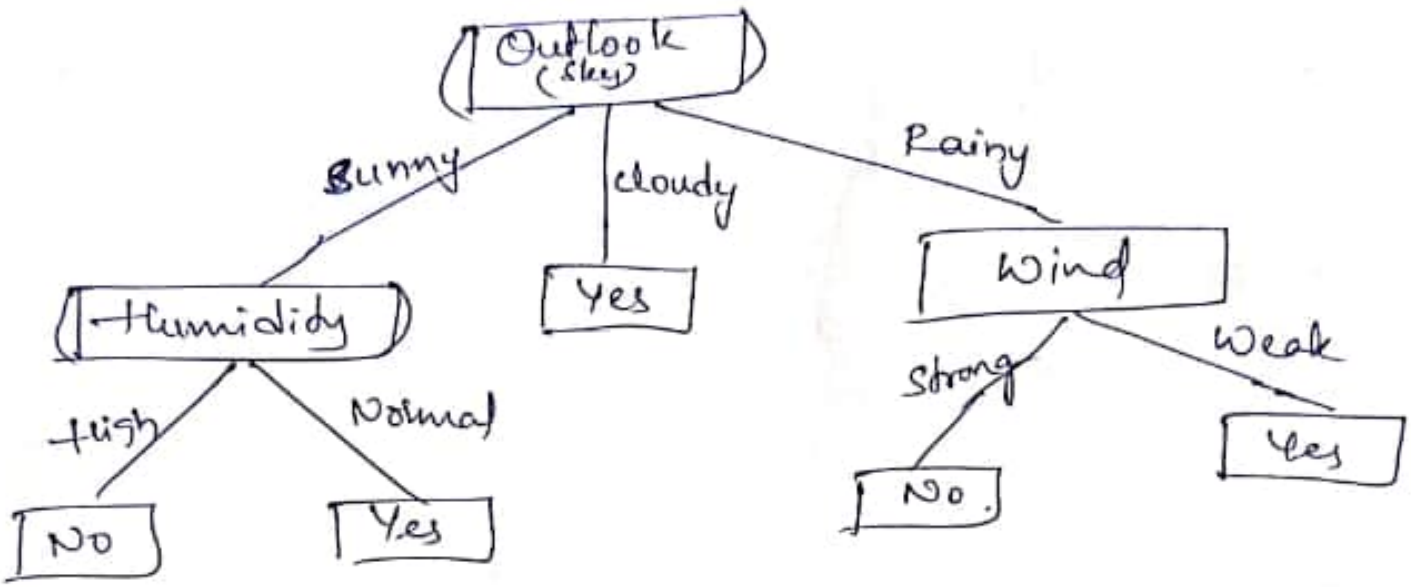


(4) Car Mileage Prediction:



EB-SBIT

Decision Tree for playing Tennis :



* Decision trees represent disjunctions of conjunctions. ie., a set of logical disjunctions (a set of if ... then rules).

⇒ (Outlook = Sunny ∧ Humidity = Normal)
∨ (Outlook = Overcast/cloudy)
∨ (Outlook = Rainy ∧ Wind = Weak).

Top-Down Induction of Decision Trees (ID3):

ID3(Examples, Target-attribute, Attributes)

Examples are the training examples.

Target-attribute is the attribute whose value is to be predicted by the tree.

Attributes is the list of other attributes that may be tested by the learned decision tree.

Returns a decision tree that classifies correctly the given Examples.

- create a Root node for the tree
- If all Examples are positive, return the single-node tree Root, with label = +.
- If all Examples are negative, return the single-node tree Root, with label = −.
- If Attributes is empty, Return the single-node tree Root, with label = most common value of Target-Attr in Examples.
- Otherwise Begin
  - A ← the attribute from Attributes that best classifies Examples.
  - The decision attribute for Root ← A.
  - for each possible value, v_i, of A,
    - Add a new tree branch below Root, corresponding to the test $A = v_i$
    - Let $Examples_{v_i}$ be the subset of Examples that have value v_i for A

P.T.O

- If Examples$_{vi}$ is Empty
  - Then below this new branch add a leaf node with label = most common value of Target_attribute in Examples.
  - Else below this new branch add the subtree ID3(Examples$_{vi}$, Target_Attribute, Attributes - {A})
- End
- Return Root.

- which attribute is selected at root node that is most useful for classifying examples?
- A good quantitative measure of an attribute is used.
- We define a statistical property, called informationgain that measures how well a given attribute separates the training examples according to their target classification
- We begin by defining a measure used in information theory, called entropy, that characterizes the (im)purity of an arbitrary collection of examples.

Given a collection S, containing positive and negative examples of some target concept, the entropy of S relative to the boolean classification is

$$\text{Entropy}(S) = -P_{\oplus} \log_2 P_{\oplus} - P_{\ominus} \log_2 P_{\ominus}$$

EB-SBIT

where $P_{\oplus}$ → fraction of +ve examples in S
$P_{\ominus}$ → " " −ve " " S

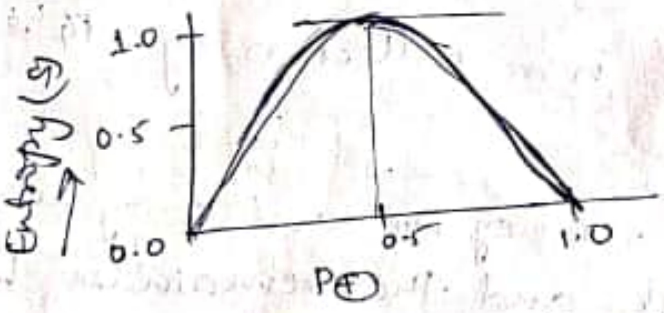
Note: $\boxed{0 \log_2 0 = 0}$.

If the target-attribute can take on c different values, then the entropy of S relative to this c-wise classification is defined as

$$\text{Entropy}(S) = \sum_{i=1}^{c} -p_i \log_2 p_i ,$$

where p_i is the proportion of S belonging to class i.

— The entropy function relative to the boolean classification, as the proportion of +ve examples, $P_{\oplus}$, varies between 0 and 1.



— If all members are +ve ($P_{\oplus} = 1$) then $P_{\ominus} = 0$, and

$$\text{Entropy}(S) = -1 \cdot \log_2(1) - 0 \cdot \log_2 0$$
$$= -1.0 - 0 = \underline{0} .$$

— Note the entropy is 1 when the collection contains an equal no. of positive and negative examples.

— If collection contains unequal no. of +ve and −ve examples, the entropy is between 0 and 1.

Eg:
Entropy [9+, 5−] $= -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right)$
$= 0.940$.

Given entropy as a measure of the impurity in the collection, we now define a measure of effectiveness of an attribute in classifying the training data.

This measure is called Information Gain, is the expected reduction in entropy caused by partitioning the examples according to this attribute.

The Information Gain, Gain(S, A) of an attribute A, relative to the collection S is defined as

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{S} Entropy(S_v)$$

Ex:- S is a collection of 14 examples [9+, 5-].

EB-SBIT

Of these 14, 6 of +ve and 2 of -ve examples have Wind = weak and the remainder has Wind = Strong.

The informn gain due to sorting the original examples by the attribute wind may be calculated as

Values (Wind) = Weak, Strong

S = [9+, 5-] — Entropy(S) = 0.940

S_{weak} ← [6+, 2-] — Entropy(S_{weak}) = 0.811

S_{strong} ← [3+, 3-]. — Entropy(S_{strong}) = 1.00.

$$Gain(S, Wind) = Entropy(S) - \sum_{v \in \{Weak, Strong\}} \frac{|S_v|}{S} Entropy(S_v)$$

$$= Entropy(S) - \frac{8}{14} Entropy(S_{weak}) - \frac{6}{14} Entropy(S_{strong})$$

$$= 0.940 - \frac{8}{14}(0.811) - \frac{6}{14}(1.00)$$

$$= 0.048$$

Entropy (Sweak) = $-\frac{6}{8}\log_2\left(\frac{6}{8}\right) - \frac{2}{8}\log_2\left(\frac{2}{8}\right)$
8[6+,2-], $= -\frac{6}{8}(-0.415) - \frac{2}{8}(-2) =$
$= \frac{2.49}{8} + \frac{4}{8} = 0.811$

- Information gain is the measure used by ID3 to select the best attribute at each step in growing the tree.

For Humidity, calculate Gain(S, Humidity), ie,
S: [9+,5-], E = 0.940

Humidity
- High → [3+,4-], E = 0.985
- Normal → [6+,1-], E = 0.592

$$\Rightarrow Gain(S, Humidity) = 0.940 - \frac{7}{14}(0.985) - \frac{7}{14}(0.592)$$
$$= 0.151.$$



⇒ Humidity provides greater inform gain than Wind, relative to the target classificaten.

Ex:- A table which consists of training examples for the target concept PlayTennis.

{D1, D2, ... D14} [9+,5-]

Outlook
- Sunny → {D1, D2, D8, D9, D11} [2+,3-] → ?
- Overcast → {D3, D7, D12, D13} [4+,0-] → Yes
- Rain → {D4, D5, D6, D10, D14} [3+,2-] → ?

$S_{sunny} = \{ D1, D2, D8, D9, D11 \}$.

$$Gain(S_{sunny}, Humidity) = E(S_{sunny}) - \frac{|S_{sunny(High)}|}{|S_{sunny}|} . E(S_{sunny(High)}) - \frac{|S_{sunny(Normal)}|}{S_{sunny}} . E(S_{sunny(Normal)}).$$

$Gain(S_{sunny}, Humidity) = 0.971 - \frac{3}{5}(0.0) - \frac{2}{5}(0.0) = \underline{0.970}$

$Gain(S_{sunny}, Temperature) = 0.971 - \frac{2}{5}(0.0) - \frac{2}{5}(1.0) - \frac{1}{5}(0.0) = \underline{0.570}$

$Gain(S_{sunny}, Wind) = 0.971 - \frac{2}{5}(1.0) - \frac{3}{5}(0.918) = \underline{0.19}$

Out of all these gains, Humidity has highest value, so it is selected for that branch.

(*). $Gain(S, Outlook) = 0.246$

$Gain(S, Humidity) = 0.151$

$Gain(S, Wind) = 0.048$

$Gain(S, Temp) = 0.029.$

Of all the four values Outlook attribute provides the best prediction of the target attribute, PlayTennis, over the training examples.

∴ Outlook is selected as the decision attribute for the root node.

EB-SBIT

$E(S_{sunny}) = -\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right)$

These steps are repeated for each new leaf node until either of the conditions are met:

(i) every attribute has already been included along this path through the tree, or

(ii) the training examples associated with this leaf node all have the same target value.

i.e., when to stop:

- no more input features
- all examples are classified the same
- too few examples to make an informative split.

⇒ which test to split on

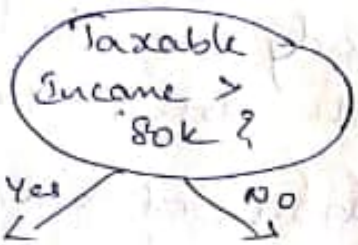
- split gives smallest error.
- with multi-valued features
  - split on all values or
  - split values into half.

Dealing with Continuous variables:

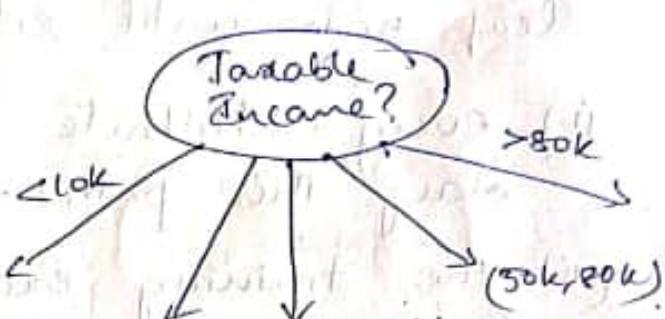
The s.o.l. is to discretise the continuous variable. for a continuous variable there is not just one place to split it, the variable can be broken between any pair of datapoints.

In general, only one split is made to a continuous variable, rather than allowing for threeway or higher splits.

Splitting of a continuous variable can be done into two halves.

Ex:- 

(or)



(i) Binary split (ii) Multi-way split

(i) For continuous attribute
accomplished by
- Dynamically defining new discrete valued attributes that partition the continuous attribute value into a discrete set of intervals.

⇒ For an attribute A that is continuous-valued, the algo dynamically create a new boolean attribute Ac that is true if A < c and false otherwise.

- Question is to select the best value for the threshold 'c'.

Ac = { true if A < c
     { false otherwise

How to choose c?

- consider all possible splits and finds the best cut.

Ex:-

Temperature	:	40	48	60	72	80	90
Play Tennis	:	No	No	Yes	Yes	Yes	No

- Pick a threshold, c, that produces the greatest inform. gain.

- Sort the examples according to continuous attribute A.
- Then identify the adjacent examples that differ in their target classification
- we can generate a set of candidate thresholds midway between the corresponding values of A.
- It is shown that value of c that maximizes the informn gain must always lie at such boundary.
- These candidate thresholds can be evaluated by computing the informn gain associated with each.

Ex:- * Two candidate thresholds, corresponding to the Temp values at which value of PlayTennis changes:

- $\left(\frac{48+60}{2}\right)$, and $\left(\frac{80+90}{2}\right)$

* compute informn gain for each of candidate attributes Temp>54 and Temp>85 and the best can be selected (Temp>54).

- Split the continuous attribute into multiple intervals rather than just two intervals based on a single threshold.

Alternative Measures for Selecting Attributes:

Classification and Regression Trees (CART):

- well-known tree-based algo used for both classification and regression.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject :

H.T. No. :
Date :
Signature of Invigilator

EB-SBIT

Classification and Regression Trees (CART): How to split attribute using Gini Index

Just like the information measure, there is another possibility to pick up the features for the split, which is known as Gini Impurity.

Aim of the decision tree is to have each leaf node represent a set of data points that are in the same class, so that there are no mismatches. This is known as purity.

If a leaf is pure, then all examples belongs to one class.

If we count the no. of data points at the node that belong to class i [$N(i)$] then it (practice) should be '0' for all except one value of i.

To choose for a split: The algorithm loops/repeats over diff. features and checks how many points belong to each class.

If node is pure then $N(i) = 0$ for all values of i except one.

So for any particular feature k, we can compute

$$G_k = \sum_{i=1}^{C} \sum_{j \neq i} N(i) \cdot N(j),$$

where $C \rightarrow$ no. of classes.

In fact, $\sum_i N(i) = 1$, then, and so

$$\sum_{j \neq i} N(j) = 1 - N(i)$$

The above equation becomes,

$$G_k = 1 - \sum_{i=1}^{c} N(i)^2$$

The information gain can then be measured in the same way, subtracting each value G_i from the total Gini impurity.

— We calculate the Gini Index of the feature given by

$$\text{GiniIndex}(a_1) = \sum_{v \in \text{Values}(a_1)} \frac{|S_v|}{|S|} \text{Gini}(S_v)$$

, where $\text{Gini}(S_v)$ is G_k.

FR-SBIT

$$\text{Gini} = 1 - \sum_{i=1}^{c} (P_i)^2$$

Ex:-

a_1	a_2	a_3	Target class
T	T	1.0	+
T	T	6.0	+
T	F	5	−
F	F	4	+
F	T	7	−
F	T	3	−
F	F	8	−
T	F	7	+
F	T	5	−

$$\text{Gini}(T) = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 0.375$$

$$\text{Gini}(F) = 1 - \left(\frac{1}{5}\right)^2 - \left(\frac{4}{5}\right)^2 = 0.32$$

$$\text{GiniIndex}(a_1) = \sum_{v \in [T,F]} \frac{|S_v|}{|S|} \text{Gini}(S_v)$$

$$\text{GiniIndex}(a_1) = \frac{4}{9}(0.375) + \frac{5}{9}(0.32)$$

$$= 0.3444$$

(b) $\text{GiniIndex}(a_2) = \frac{5}{9}(0.48) + \frac{4}{9}(0.5) = 0.4889$

Giniindex (a_1) = 0.3444

Giniindex (a_2) = 0.4889.

The smaller the Giniindex, the splits of the attribute is purer.

Hence, a_1 attribute is the best split attribute.

This way the decision tree is constructed using Gini impurity

Ensemble learning.

Ensemble learning considers multiple classifiers and combines the outputs of multiple classifiers inorder to get better prediction or classification accuracy.

In some situations, where the classifier o/ps are independent of each other and make errors in an independent manner, it is possible to combine the o/ps of several classifiers getting the resulting classifier which is better than any of the constituent classifiers.

For a given data points, several/multiple classifiers give multiple outputs which can be same or different or few may be same and few may be different.

We need to have a way to combine the decisions or outputs by the multiple learners. For this, we need to generate a group of base learners.

These different group of learners can be different and may be using different algos. ie, there is a learner using decision trees, or neural n/ws or svm, etc.

So, we can feed the training examples

to different algorithms and combine the o/p

In general, only one classifier can use at a time (but may not). If we different classifiers, then use different hyper parameters and may have different representations or different modalities or may use diff. training sets.

* Learning algorithms may have high variance when applied on different subsets of data and may produce different models as o/p.

- So try to use learning algos which are weak and have high variance can be fed different training set, and as a result different models are produced which can be combined.

- ~~What about~~ are hypo parameters? They are in Neural N/ws, there are certain parameters to be set like no. of hidden layers (i.e., topology), no. of hidden nodes in each hidden layer, and weights and bias and so on. So,

FR-SBIT

By giving different values to these parameters, we can get different models for the given neural n/w.

In decision trees, we try to come up with the strategy for deciding the attribute to split ~~on~~ at a particular node.

- For splitting we can use diff heuristic methods/functions like entropy, gini index, etc. we could modify the decision tree algo using different fns so that diff learners do not split on the same attribute, and force the learners to form different decision trees.

So, these are the ways we can get a group of learners.

- It is found that no one learner is good learner for many problems.
- we may have multiple learners, may not be good enough, but infact they may be weak but they may make independent errors.
- So, all the learners which are different may work well in different parts of the data, and by combining these weak learners it is possible to get a strong learner.

For example, let us have 2-class classification which is having two classes positive and negative classes.

Suppose if we take some learners, and each learner has an error (ϵ) but the ϵ has to be greater than 0.5.

If $\epsilon = 0.5$ then there is no informative learner so, Error must be greater than 0.5.

i.e., $\boxed{\epsilon > 0.5}$.

In ensemble learning, we want the different constituent learners in which each will have an accuracy that accuracy may not be very high but it must be greater than 0.5:

- If other conditions are met, it is not required that the accuracy ~~must be~~ has to be higher than 0.5 but it has to be higher than 0.5, need not be very high.

FR-SBIT

Problem of overfitting

Decision tree has: ... it is actually overfitted..

weak learners

1) Low bias – fits very well to the training data
2) High variance – less predictive capability, when new test data is fed.

To reduce such types of problems, in decision tree, we have two approaches:
① Boosting ② Bagging (Bootstrap aggregating).

- Bagging = "Bootstrap Aggregation"
  — Draw N items from X with replacement
- Derived learners with high variance (unstable)
  — Decision trees and ANNs are unstable
  — KNN is stable
- Use bootstrapping to generate L training sets and train one base-learner with each.
- use voting.

To use an ensemble of learners and be able to combine the learners and get better accuracy, what is needed is a set of learners which would make independent errors.

In order to force the learners to make independent errors, we can make them use different samples, diff parameters of the algorithm,

or make them to use different algorithms, and output can be combined some form of voting.

① In Bootstrap aggregation, first of all the samples are generated such that the samples are different from each other.

Given a training set D which is having a size of n,

Bagging generates m new subsets of the training sets. Lets refer them as Di, each of size n' by using the sampling with replacement (Ex: Fishes catching).

FR-SBIT

By this replacement, some observations may be repeated in each Di.

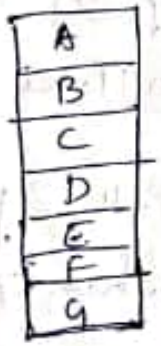
If n'=n, then for large n the set Di is expected to have the fraction $(1-\frac{1}{n})^n = (1-\frac{1}{e})$ (~63.2%) of the unique samples of D, the rest being duplicates.

This kind of sample is called bootstrap sample.

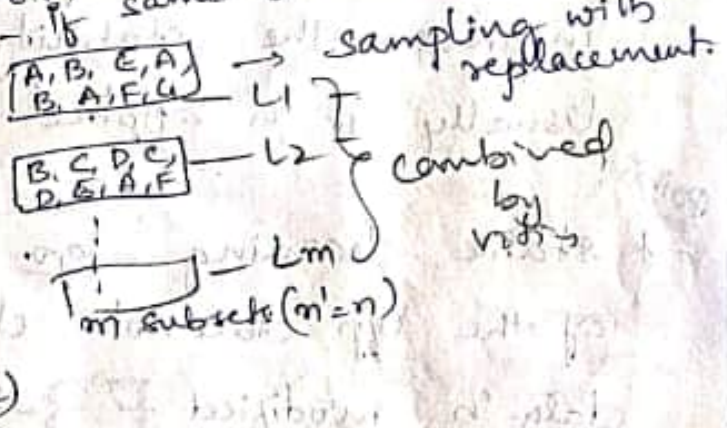
- Each bootstrap sample is independent from others as it doesn't depend on previous chosen samples when sampling.

- Then m no of learning models are fed with these D_i samples and outputs are combined by averaging for regression or voting for classification

Ex:-







(+) If n is not large (i.e., size), splitting it into smaller m diff subsets, also will be small.

Also, if we use a learner on a small training dataset, the learner can overfit, it can have high variance and will not be generalized well.

And will not be able to have fully independent data, so we can sample randomly from the data so that some examples may be repeated [Do it by sampling instead of dividing] (Sampling with replacement).

- Some examples may not appear in these samples.

As ensembles require the learners with high variance, By combining these on learners, we can reduce these variance (by $\frac{1}{k}$).

So, this bagging technique is used to improve the stability and accuracy of ML algos. Usually it is applied on the Decision Tree.

Points:

* stable learning algo is one in which the prediction of the o/p does not change much when the training data is modified slightly

FR-SBIT

* stable learners irrespective of what data we give, the o/p will be similar ie, they have low variance

* learners with high variance are unstable.

- Decision Tree, NN are unstable learners (given diff data give diff models).

* - k-nearest neighbour is more stable.

- So only unstable learners are used in bagging

Boosting:

It is a technique (algorithm) used for reducing bias.

- It improves the stability and accuracy of algos.
- Usually (Typically) used for converting weak learners to strong learners.

* It is an iterative procedure. Adaptively changes the distribution of training data.

* Initially, all N records (instances) are assigned equal weights (a equal probability is assigned)

- we choose D_1 sample based on it's uniform probability distribution.
- Now on D_1 train a learner (weak learner) (L_1)
- This learner L_1 is applied on original instances ie., D.

Ex: - if this learner gives 100% accuracy then it is a strong learner, but in general it doesn't happen is the learner labels some of these instances in D correctly and some incorrectly labelled.

- we want next learner to do well on those instances where the current learner has not done well (not classified correctly)

- Now change the probability distribution.
- Specifically, we will increase the probability of these wrongly classified instances, and those which are correctly labelled, we will proportionately reduce the probability so that sum of all probabilities is 1.
- we will hide the probability of the instances that were not correctly classified by the 4 learner.
- Now we will have the same dataset, but the probability distribution of will change, ~~of the samples~~.
- Now sample D_2 according to this probability distribution and we get a sub sample and is more likely to contain the instances that are misclassified by the previous learner.
- Apply another learning algorithm on D_2 and get a learner L_2.
- Now again apply L_2 on these training instances (D with adjusted probabilities), and find out which ones D_2 does error i.e. incorrectly classified.
- Again increase the probability of wrongly classified beyond the current probability and reduce the probability

- Again find D_3 on which learner d_3 is trained, and will be repeating the same process until we get the expected accuracy or specific no. of learners have trained.
- we get different data different learners.
- we will combine these different o/ps of the different learners by voting.
- while voting we will assign a weight/strength to each learner and weight is related to how good the learner was.
- This is the basic boosting algorithm.

---

* Initially all training instances have equal weights.
  - weights change at the end of the boosting round.

* On each iteration t:
  - weight each training example by how incorrectly it was classified
  - learn a hypothesis (learner): h_t
  - A strength for this hypothesis (learner): α_t.

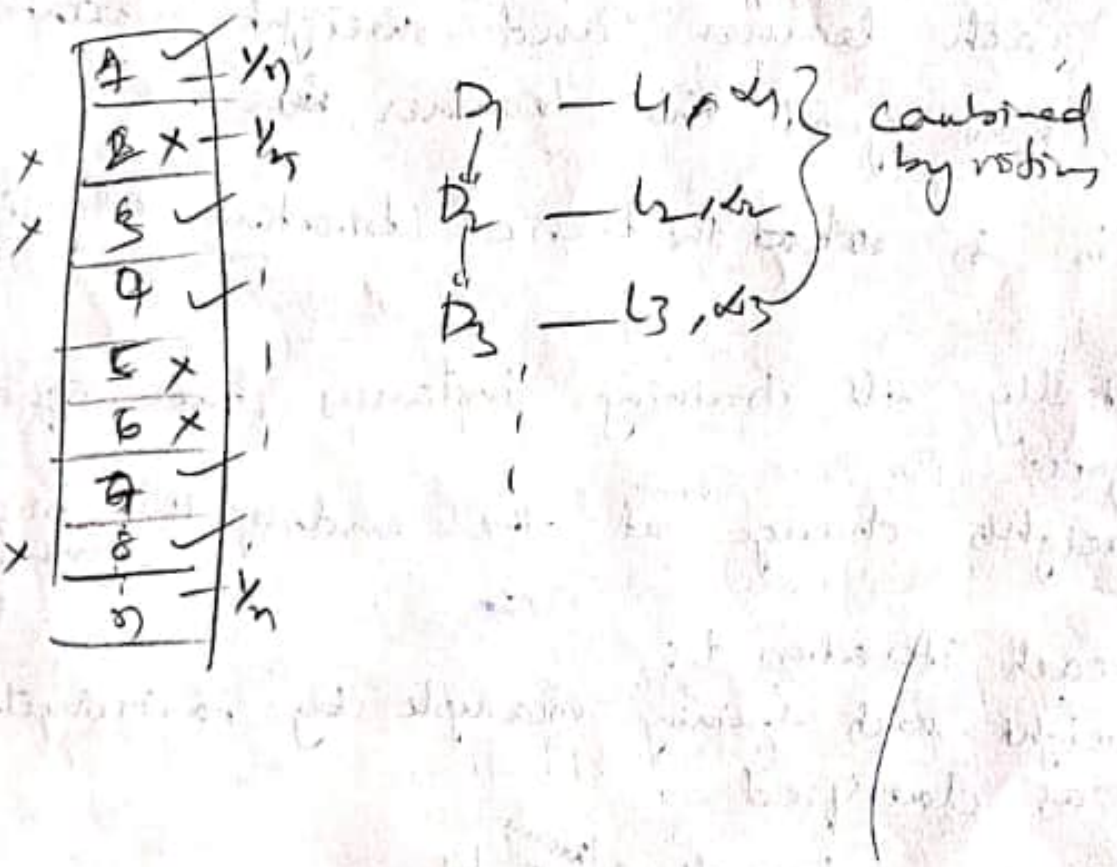
* final classifier:
  - A linear combination of the votes of the different classifiers weighted by their strengths.

* weak learners ≠
  - → P(correct) > 50%, but not necessarily much better.

this strengths is used to decide the weight of the learners voting.

The final classifier is the linear combination of this different learning hypotheses (learners) weighted by there strengths (weights).

What we require is that the individual learner must be a weak learner, Ie, they are unstable learners, but they must have more than 50% accuracy for a 2-class problem.



Adaboost : [Adaptive Boosting]

It is a common boosting algorithm. We have seen that boosting algo can convert a weak learner (algorithm) to a strong learner.

Adaboost takes a sample as input, say 'S'.

- It is mainly used for binary classification, although it can be generalized to multiple classes.

And this sample contains the examples as $\{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\}$, where

x is the vector of the i/p attribute and
y is the o/p attribute

- we have a weight for each training example initially, $D_0(i)$ is the weight of ith training example

$$D_0(i) = \frac{1}{m} \quad \forall i = 1 \text{ to } m$$

- Initially at time '0', all the examples have equal weight (probability), which is $1/m$, and we use an algorithm.
- we do T no. of iterations.
- In the first iteration, we construct S_1 on $x_1, x_2, \ldots, x_m$ by sampling from this S, using the distribution D_0.
- Now we update $D_1(i)$ for those i where h_1 incorrectly classifies [$\because h_1$ is learning model created on D_1].

— Increase $D_1(i)$ for the examples on which L_1 wrongly labelled (classified)

From $D_1(i)$, we sample S_2, and from S_2 by running the algorithm we get L_2 learner.

when L_1 is applied on ~~original data~~ S_1, we get an error as ϵ_1,

when L_2 is applied on S_2 (original data), error is ϵ_2, L_3 produces L_3 with error ϵ_3.

So ϵ_t is the error over the current sample S_t, not on the original sample.

this is repeated for some no. of iterations.

steps : $D_1(i) = \frac{1}{m}$

for $t = 1 \ldots T$

— Train the learners using D_t $\boxed{h_t : X \to \{-1, 1\}}$

— Get a weak classifier, let's call it h_t

— we choose α_t as the weight associated with h_t, based on how much h_t is accurate.

[∵ how

— update the distribution so that we get

$$D_{t+1}(i) = \frac{D_t(i) \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))}{Z_t}$$

$$\boxed{Z_t = \sum_{i=1}^{m} D_t(i) \cdot \exp(-\alpha_t y_i h_t(x_i))}$$

where Z_t is the normalization factor, is chosen s.t the sum of probabilities over i is equal to 1.

$D_0(i) = \frac{1}{m}$ this is initial weights
$D_1(i)$ is updated weights
$S_1 = L_1, \epsilon_1$ error
$S_2 = L_2, \epsilon_2$ error
$S_3 = L_3$ "

Output : final classifier,

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$$

⇒ $h_t(x)$ are different classifiers weighted by the α_t.

⇒ And the total is computed - If the total is +ve then the class is +ve, and if the total is -ve, then the class is -ve.

To compute α_t, first we need to compute ϵ_t. Where ϵ_t is the error of the hypothesis h_t with the current sample

$$\epsilon_t = \sum_{i=1}^{m} D_t(i)\, \delta\left(h_t(x_i) \neq y_i\right),$$

where $D_t(i)$ → the probability of the ith training example in the sample.

And $\delta(h_t(x_i) \neq y_i)$ → is 1 if x_i is misclassified
→ is 0 if otherwise.

and

- we need to take the no. of misclassifications taking the weighted sum of no. of misclassifications so weighed by the probability so that it is ϵ_t, and α_t is computed as

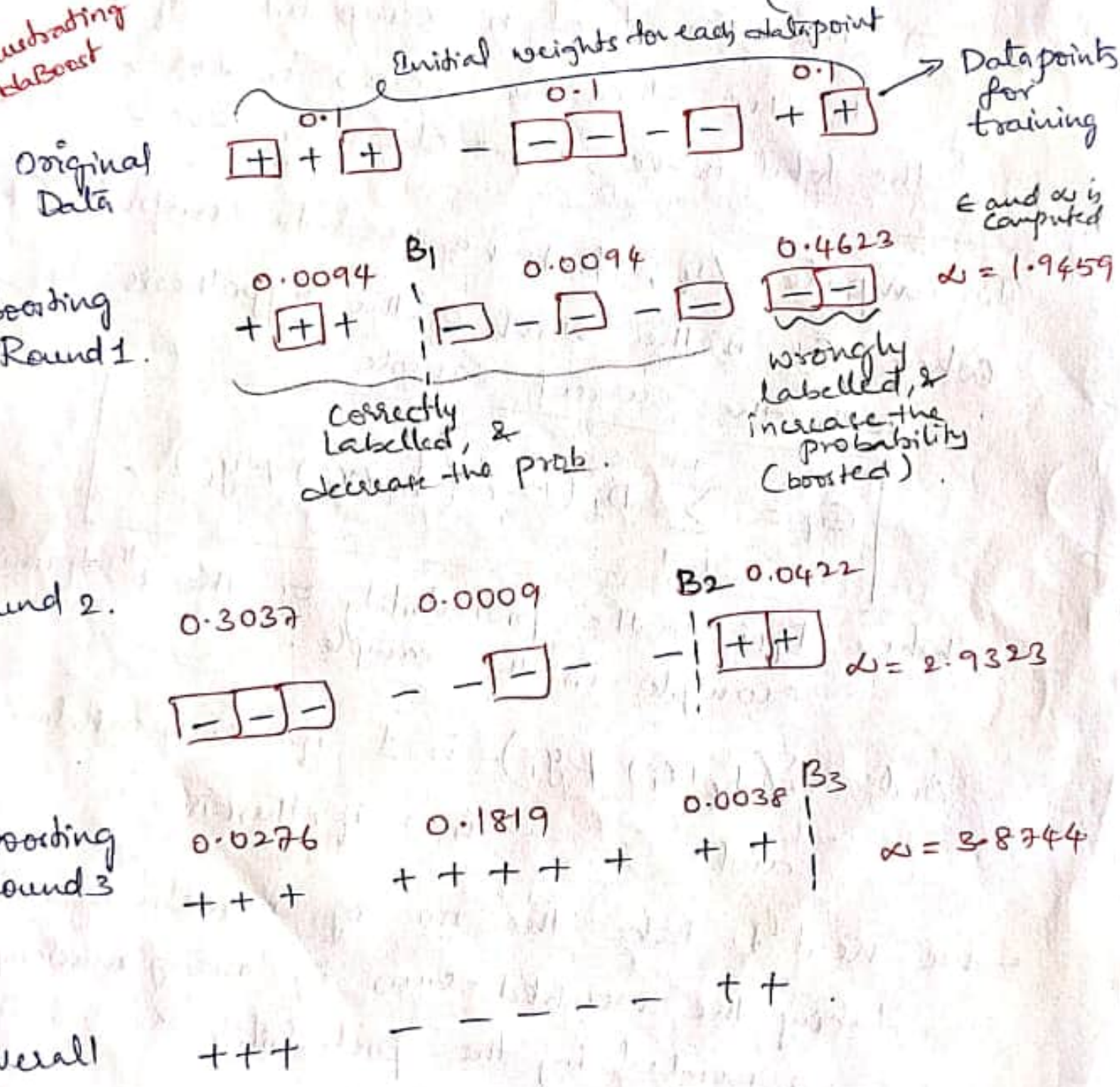
$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

where ϵ_t is the measure of the error and

α_t is the accuracy of the particular hypothesis h_t.

— In the ensemble, when we are combining the votes of the diff hypothesis, α_t is considered. (and is based on α_t).

Illustrating AdaBoost



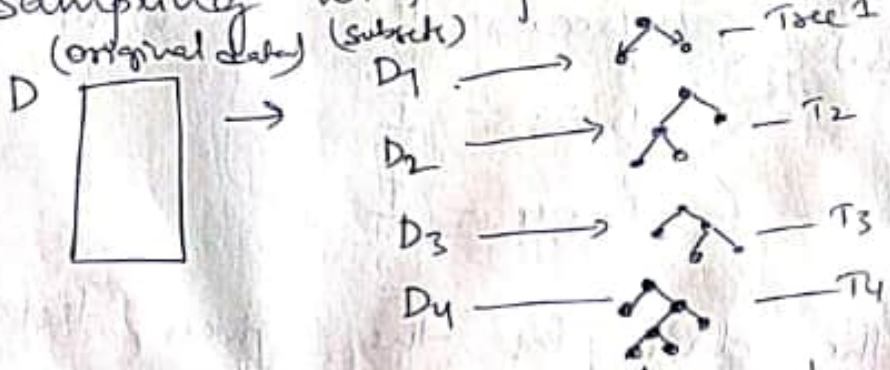
Finally 3 classifiers are combined using α_1, α_2 and α_3 and classifies training examples correctly.

Random forests:

505/17, 517, 518 Dec

One of the algorithm which use bagging is decision tree classifier, is Random forest, and the principle of drawing the tree will be same but we will change how we build the tree.

- From the original training data we need to perform the bootstrap, ie., we divide the training data into multiple training data (subsets) using sampling with replacement.



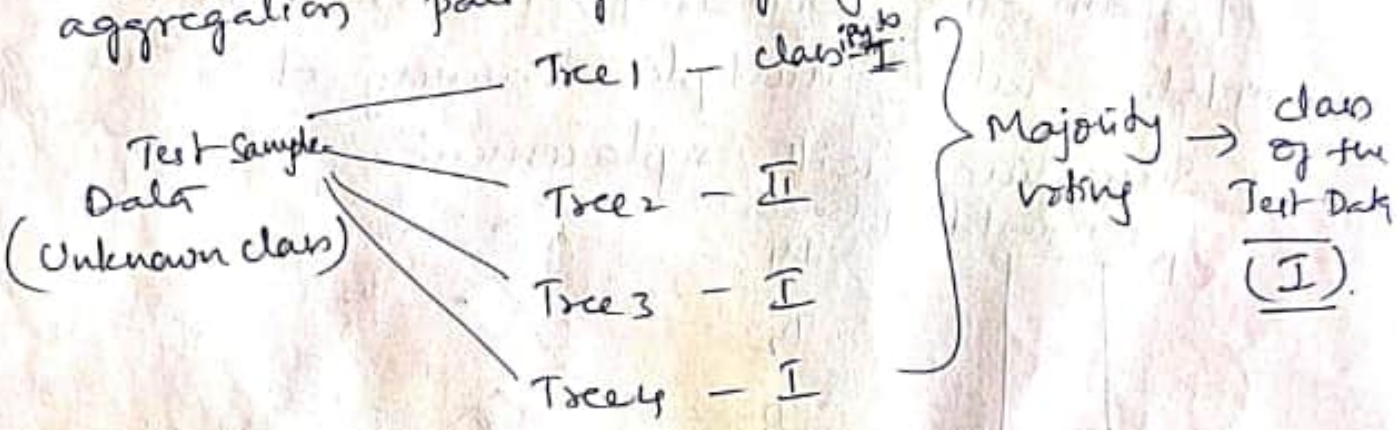
For each subset ie., subsample, create a decision tree. while creating this decision tree, we are not considering all the predictors. (independent vars or features or attributes to predict target)

Only a random subset of the features are selected from the subsample and find which ~~best~~ feature is best to split and build a decision tree.

Like this for each subsample, build a decision tree taking different subset of the features, we do not take all the features. (other subset)

- All these trees are generated from bootstrap data i.e., we use a subset of the data as well as they are decorrelated (ie., we used m number of p predictors.

This is how you create a forest, ie., forest of decision trees. This forest is used to classify the test data, ie., where the aggregation part of bagging comes.



- Predict the class for the given test sample using all the decision trees.
- Then apply the principle of majority vote to decide the class of the sample.

Different ways to Combine classifiers:

- Bagging ensures that different classifiers see different data since they are applied on different samples of the data.
- In Boosting, the data stays the same, but the data points get different weights (probabilities) according to how well the previous classifiers have performed.
- Important point in ensemble method is how it combines the o/p's of diff. classifiers:
  * Bagging takes the majority vote
  * Boosting takes a weighed vote.
- There are other alternatives to these methods
- ~~Bagging~~, for classification, algos produce an o/p where all classifiers agree, or more than half of them agree, so simply take the most common o/p ie, by majority voting.

Assuming that each individual classifier has a success rate of p, the probability of ensemble getting correct answer is a

(no. of classifiers is odd).

binomial distribution of the form:

$$\sum_{k=\frac{T}{2}+1}^{T} \binom{T}{k} p^k (1-p)^{T-k}$$

where T is no. of classifiers.

- If $P > 0.5$, then this sum approaches 1 as $T \to \infty$.
- If no. of classifiers is 100, then the probability of the ensemble being correct gets close to 1.
- Even <50% chance of success for each individual classifier, the ensemble works very well.
- For regression problems, instead of taking the majority vote, it takes the mean of the o/ps.

However, mean is heavily affected by outliers, with the result that the median is more common average to use. This is what produces the bragging algorithm, which is meant to imply 'robust bagging'.

— There is an algorithm that is used to combine the classifiers, known as the "mixture of experts."

⇒ Inputs are fed to the n/w, and each individual classifier makes an assessment (produces an o/p).

⇒ These o/ps are then weighted by the relevant gate, which produces a weight w using the current i/ps and this is propagated further up the hierarchy.

⇒ The most common version of mixture of experts works as follows:

Mixture of experts algorithm:

- for each expert:
  - calculate the probability of the input belonging to each possible class by computing (where the w_i are the weights for that classifier):

$$O_i(x, w_i) = \frac{1}{1 + \exp(-w_i \cdot x)}$$

- For each gating n/w up the tree:
  - compute:

$$g_i(x, v_i) = \frac{\exp(v_i x)}{\sum_i \exp(v_i x)}$$

- pass as input to the next level gates (where the sum is over the relevant inputs to that gate).

$$\sum_k q_j g_j \cdots$$

This network is trained using an Expectation-Maximization (EM) algorithm [a statistical approximation algorithm].

Probabilistic Learning

In some classifiers (Bayes, NN, D.Trees) the data had target labels and so we used supervised learning, i.e., learning the probabilities from the labelled data.

If we have the same data, but without target labels then unsupervised learning is required.

In many applications, to represent the data distributions we considered a Gaussian distributions, i.e., a particular data distribution can be approximated by Gaussian distribution.

Advantages of Gaussian distributions:

- It is very closer to natural distributions.
- Mathematical manipulations for Gaussian functions is easy
  - * to determine derivative of G. func
  - * to determine the nth order derivative of G.f
- In case of complex data set, or may be the distribution, it is not possible to rep this data set by a single G. Gaussian.

For this, we need to consider multiple Gaussians, i.e, mixture of Gaussians.

In case of a Gaussian distribution, or Gaussian density, it has two parameters, one is mean and other is variance. [one-dimensional distribution]

In multidimensional case, the parameters considered are ① mean vector, and ② covariance matrix.

— These parameters values are determined by some estimation techniques like maximum likelihood.

— In case of a Gaussian Mixture Model, we cannot apply the maximum likelihood estimation technique because it cannot give a close form of solution.

So, for this, we need to apply an iterative algorithm which is popularly known as expectation maximization algo.

—

Let us consider a Gaussian distribution, so, this has two parameters : mean & variance and the Gaussian density is given as,

$$N(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma} e^{-(x-\mu)^2/2\sigma^2}$$

for univariate Gaussian density

for multivariate normal distribution or Gaussian distribution, we can write like,

$$N(\bar{x}, \bar{\mu}, \Sigma) = \frac{1}{(2\pi)^{d/2} \cdot |\Sigma|^{1/2}} \times \exp\left(-\frac{1}{2}(\bar{x}-\bar{\mu})^T \Sigma^{-1} (\bar{x}-\bar{\mu})\right)$$

↳ multivariate normal density

In order to estimate the parameters of the distribution, we consider the maximum likelihood estimation

If we have some data points like below, this distribution can be represented by a Gaussian function



If multidimensional case is considered, we may get this type of ellipse and a centroid that means, it is the mean of this Gaussian.



So, to determine the values of these parameters, we require a ML technique the maximum likelihood estimation technique.

- So we need to consider the log of Gaussian distribution in case of maximum likelihood estimate
- and take the derivative and equate it to zero '0'.

ie,

$$\ln P(\bar{x} | \bar{\mu}, \Sigma) = -\frac{d}{2}\ln(2\pi) - \frac{1}{2}\ln|\Sigma| - \frac{1}{2}(\bar{x}-\bar{\mu})^T \Sigma^{-1}(\bar{x}-\bar{\mu})$$

Take the derivative of this log,

$$\Rightarrow \frac{\partial \ln P(\bar{x}|\bar{\mu}, \Sigma)}{\partial \bar{\mu}} = 0 \quad \text{and} \quad \frac{\partial}{\partial \Sigma} \ln P(\bar{x}|\bar{\mu}, \Sigma) = 0.$$

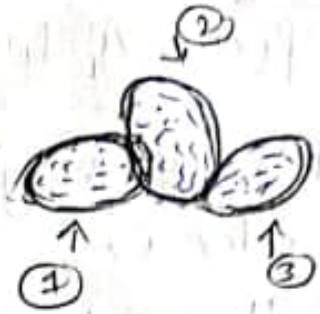
$$\Downarrow \qquad \Downarrow$$

$$\bar{\mu}_{ML} = \frac{1}{N} \cdot \sum_{n=1}^{N} \cdot \bar{x}_n \qquad \Sigma_{ML} = \frac{1}{N} \sum_{n=1}^{N} (\bar{x}_n - \bar{\mu}_{ML})(\bar{x}_n - \bar{\mu}_{ML})^T$$

where, $N \rightarrow$ no. of samples or datapoints.

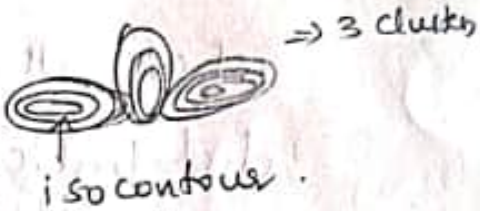
Gaussian Mixture Model:

why we need a mixture model, let us consider a data set



→ A single Gaussian distribution cannot be considered for to represent this data distribution.

— So, we have to consider multiple Gaussians, in this case, we may consider three Gaussians to represent data distribution.



— The distance from the mean is same for isocountours.

— The entire data distribution is represented by a mixture model, the mixture of three Gaussians.

— In one dimension also we can show this one

1D.



This is the concept of the mixture model.

Ex:- consider two Gaussians.

for $f_0(x) = N(x, 2, 2)$ and $f_1(x) = N(x, 10, 25)$

$\pi = [.5, .5]^T$



GMM is represented mathematically as,

$$p(x) = \pi_1 f_1(x_k) + \pi_2 f_2(x_k) + \cdots + \pi_k f_k(x_k)$$

this density can be represented as a mixture model i.e, the linear superposition of Gaussians.

Here $k \rightarrow$ no. of Gaussians

Mixture model: It is the weighed sum of the mixtures of probabilistic distribution functions where the weights are determined by a distribution, given by π.

$\Rightarrow$ i.e., the linear superposition of Gaussians

where $\sum_{k=1}^{K} \pi_k = 1$

It can also be written as

$$p(x) = \sum_{k=1}^{K} \pi_k \cdot f_k(x)$$

$$\Rightarrow p(x) = \pi_1 N(x | \mu_1, \Sigma_1) + \pi_2 N(x | \mu_2, \Sigma_2) + \cdots + \pi_k N(x | \mu_k, \Sigma_k)$$

$$\Rightarrow p(x) = \sum_{k=1}^{K} \pi_k \cdot N(x | \mu_k, \Sigma_k)$$ ← Mixture model.

Let us discuss the Expectation-Maximization model.

First

$$P(x) = \sum_{k=1}^{K} \pi_k N(x \mid \mu_k, \Sigma_k)$$

with constraint $\sum_{k=1}^{K} \pi_k = 1$.

where $K \rightarrow$ K no. of Gaussians,

$\pi \rightarrow$ Mixing coefficients / weights

we need to consider some conditions like

Normalization and Positivity

$0 \leq \pi_k \leq 1$ $\quad$ $\sum_{k=1}^{K} \pi_k = 1$

To determine the log likelihood, we have

$$\ln P(X \mid \underbrace{\mu, \Sigma, \pi}_{\text{3 parameters}}) = \sum_{n=1}^{N} \ln P(x_n) = \sum_{n=1}^{N} \ln \sum_{k=1}^{K} \pi_k \cdot N(x_n \mid \mu_k, \Sigma_k)$$

So, these parameters can be calculated by considering EM algorithm.

We can think of mixing coefficient (π) as the prior probabilities for the components.

The basic idea of EM algorithm is that,

⇒ it is easier to add extra variables that are not actually known (called hidden or latent vars, ie; responsibilities) and then to maximize the function over those vars.

— for a given value of X, we can evaluate the posterior probabilities, and which is called responsibilities, ie; class prior probability

Baye's Rule

$$\gamma_k(x) = P(k|x) = \frac{P(k) \cdot P(x|k)}{P(x)}$$

$\gamma_k(x)$ ← latent variable; $P(k|x)$ ← posterior probability; $P(k)$ ⇒ class prior probability; $P(x|k)$ → class conditional density; $P(x)$ → unconditional prior

$$= \frac{\pi_k \cdot N(x|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \cdot N(x|\mu_j, \Sigma_j)}$$

where $\pi_k = \frac{N_k}{N}$, where N_k = no. of points assigned to cluster K.

N = total no. of data points.

EM algorithm is an iterative optimization technique which is operated locally

There are two steps : one is Expectation or Estimation step (E-step)

Second : Maximization step (M-step)

In the GMM EM Algorithm;

Initial-step;

1. Initialize the mean μ_j, covariance Σ_j, and mixing coefficient π_j

→ Evaluate the initial values by likelihood, ie, randomly chosen values from the dataset

2. E-step; Evaluate the responsibilities using the current parameter values

$$\text{gamma}\ \gamma_j(x) = \frac{\pi_k\, N\left(x \mid \mu_k, \Sigma_k\right)}{\sum_{j=1}^{k} \pi_j\, N\left(x \mid \mu_j, \Sigma_j\right)}.$$

M-step : Reestimate the parameters using the current responsibilities (μ_j, Σ_j, π_j)

$$\mu_j = \frac{\sum_{n=1}^{N} \gamma_j(x_n) \cdot x_n}{\sum_{n=1}^{N} \gamma_j(x_n)}$$

$$\Sigma_j = \frac{\sum_{n=1}^{N} \gamma_j(x_n)\,(x_n - \mu_j)\,(x_n - \mu_j)^T}{\sum_{n=1}^{N} \gamma_j(x_n)}$$

$$\pi_j = \frac{1}{N} \sum_{n=1}^{N} \gamma_j(x_n)$$

Repeat these E-step and M-step until the convergence.

4. For testing the convergence, we can evaluate the log likelihood by

$$\ln p(X|\mu, \Sigma, \pi) = \sum_{n=1}^{N} \ln \left\{ \sum_{k=1}^{K} \pi_k N(x_n | \mu_k, \Sigma_k) \right\}$$

If there is no convergence, return to step (2).

- Also convergence is tested using the two successive iterations.

If two successive iterations, if there is not much change in the parameter values (μ, Σ, π) then we can stop the algorithm.

(or) if there is no change in the log likelihood in two successive iterations then also we can stop the algorithm. (condition for the convergence).

Nearest Neighbour Methods:

If we don't have a model that describes the data, then the best thing to do is to look at similar data and choose to be in the same class as them.

Ex:- Dance moves in the nightclub
— first closest person, then few more people.

this is what the idea behind nearest neighbour methods:

— All the datapoints are positioned within the i/p space, so we need to work out which of the training data are close to it.
→ This requires computing the distance to each datapt in training set, which is expensive: computing d subtractions and d squarings.
⇒ so has to be done $O(N^2)$ times.

— Then identify the k nearest neighbours to the test point, and the set the class of the test point to be the most common one out of those for nearest neighbour.

— The choice of k is not trivial:.

- If k is too small, the NN methods are sensitive to noise.

→ If k is too large, and the accuracy reduces as points that are too far away are considered.

Lazy Learning Algorithm; (another name)

→ Training phase: save the examples (store in Data structure so the search gets faster)

Prediction time: (training time) Get the test data point (instance), x_t.

one nearest neighbour:
- Find the training example (x_1, y_1) that is closest to x_t.
- Predict y_t as the output y_t.

- Find the k training examples $\{(x_1, y_1)(x_2, y_2) \cdots (x_k, y_k)\}$ which are closest to x_t.
- Predict y_t as the output y_t.

It depends on weighting doing classification or regression problem

classification: Predict the frequent class which is the majority among the $\{y_1, y_2, \cdots, y_k\}$

Regression: we get different numerical o/ps $\{y_1, y_2 \cdots y_k\}$ and we predict by taking the average of $\{y_1, y_2, \cdots y_k\}$.

- Improvements:
  - * weighting examples from the neighbourhood (kernel)
  - * measuring closeness (Euclidean distance)
  - * find close examples in a large training set quickly.

- As the no. of dimensions grows, the computational costs also gets higher.
  - There are sets of methods such as KD-Trees, which computes in $O(N \log N)$ time.
- As the no. of dimensions increases, so the distance to other datapoints tends to increase.
  - the datapoints might be close in some dimensions, but a long way in others.
  - Methods for dealing with these problems, known as adaptive nearest neighbour methods.

Nearest Neighbour Smoothing:

NN methods can also be used for regression by returning the average value of the neighbours to a point.

Most common methods known as kernel smoothers, and they use a kernel (weighting function between pairs of points) that decides how much weight to put onto the contribution from each datapoint according to its distance from the input.

We use two kernels that are used for smoothing.

These both kernels are designed to give more weights to the datapts that are closer to the current input, with the weights decreasing smoothly to zero as they pass out of the range of the current i/p, with the range specified by a parameter λ. They are

① Epanechnikov quadratic kernel;

$$K_{E,\lambda}(x_0, x) = \begin{cases} 0.75\left(1 - (x_0 - x)^2/\lambda^2\right) & \text{if } |x - x_0| < \lambda \\ 0 & \text{otherwise} \end{cases}$$

② Tricube kernel;

$$K_{T,\lambda}(x_0, x) = \begin{cases} \left(1 - \left|\frac{x_0 - x}{\lambda}\right|^3\right)^3 & \text{if } |x - x_0| < \lambda \\ 0 & \text{otherwise} \end{cases}$$

- Picking λ requires experimentation. Large values average more datapts, and therefore produce lower variance but at the cost of higher bias.
- Generally take the values of λ of 2 and 4, etc

— Efficient Distance computations : KD-tree

For finding nearest neighbours, the data structure used is KD-Tree.

It reduces the cost of finding a nearest neighbour to $O(\log N)$ for $O(N)$ storage.

The construction of the tree is $O(N \log^2 N)$, with much cost in the computation of the medians, which naive algo requires a sort and is therefor $O(N \log N)$.

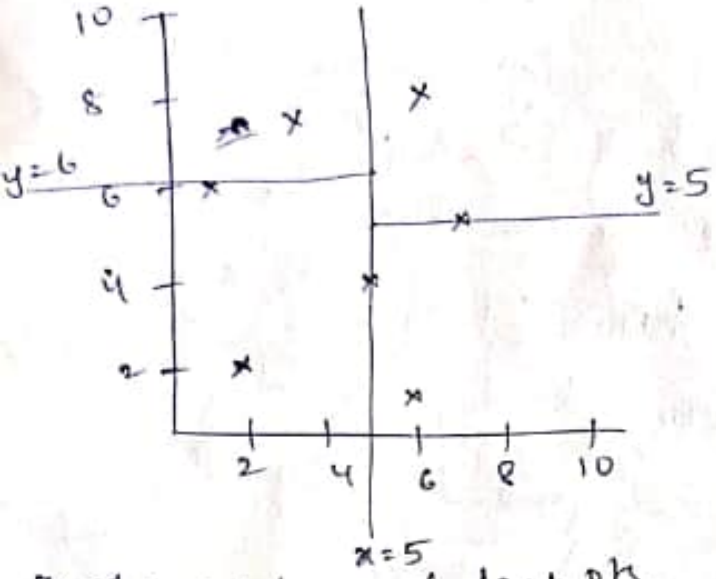
- Create a binary tree by choosing one dimension at a time to split into two, and placing the line through the median of the point coordinates of that dimension.

— Making the binary tree : we identify a place to split into two choices, left and right, and then carry on down the tree.

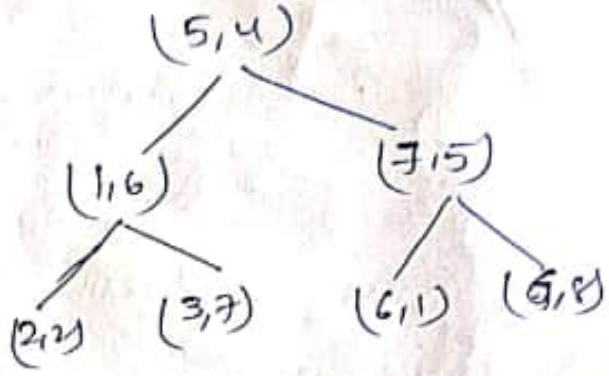
— One dimension is split in each step, and the position of the split is found by computing median of the pts that are to be split in that one dimension, and put the line there.

— The choice of which dimension to split alternates through the diff. choices, or it can be made randomly.

Ex:- Datapoints: (5,4), (1,6), (6,1), (8,5), (3,7), (5,8).



(a) The splits and leaf pts found by the KD Tree



(b) The KD-tree that made the splits.

- Searching this tree is same as binary tree.
- Starting at the root of the tree you recurse down the tree comparing just one dimension at a time until you find a leaf node → that is in the region containing the test point.

Ex:- (3,5) test point, finds (2,2) as the leaf for the box that (3,5) is in.

But looking plot, we see that this is not the closest point at all, so we need to some more work.

- label the leaf as a potential nearest neighbour, and compute the distance between test point and this pt, since any other pt has to be closer,
- check any other boxes that could contain something closer.

- from the plot we see that (3, 2) is closer, and that is label of the leaf for the sibling box to the one that was scanned.

(*) Algorithm needs to check the sibling box.

- If we take (4.5, 2) as a test point then (6,1) is closer.

(*) So, checking the sibling is not enough, also we need to check the siblings of the parent node, together with its descendants (cousins of the first pt)

Distance Measures:

(1) Euclidean distance:

Given (x_1, y_1) and (x_2, y_2), this Euclidean distance is given as

$$d_E = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

(2) City-block or Manhattan distance:

$$d_C = |x_1 - x_2| + |y_1 - y_2|$$

(3) Minkowski metric:

$$L_k(x, y) = \left(\sum_{i=1}^{d} |x_i - y_i|^k \right)^{1/k}.$$

if k=1, we get city-block (L_1 Norm)

if k=2, we get Euclidean distance (L_2 Norm)

(*) → The mean minimizes the Euclidean distance (the sum of squared distance)

→ and the median minimizes the L_1 metric.

— Also we have another distance measure named Mahalanobis distance.

- plenty of other metrics to choose, depending upon the dataspace.

— Suppose we want our classifier to recognize images (faces).
- we take a set of photos of faces and use the pixel values as features.

— use the nearest neighbour algo to identify each face.

— Even if we ensure that photos are taken fully face-on, there are still a few things that should be considered:
⇒ slight variations in the angle of the head/camera
⇒ different distances between the face and the camera (scaling)
⇒ diff. lighting conditions.

All these can change the results and can make a difference.

⇒ These can be fixed in preprocessing, but there is another alternative:
- use a different metric that is invariant to these changes. (don't vary as they do)

- The idea of invariant metrics is to find measures that ignore changes that we don't want.
  * if we want to rotate shapes around and still recognize them, you need a metric that is invariant to rotation.

- A common invariant metric in use for images is the tangent distance, works well for small rotations and scalings.

: Unsupervised learning :

- Where there is no information about the correct o/ps available, then the algorithm, is left to find some similarity between the diff inputs for itself. This is what unsupervised learning.
- We can't perform the regression, as we don't know the o/ps for any points, so we can't guess what the function is.

Then Can we do classification ?

⇒ The aim of the classification is to identify the similarities between the i/ps that belong to the same class.

⇒ There isn't any information about the correct classes but the algorithm can find similarities between i/ps inorder to cluster i/ps that are similar together.

⇒ This might perform classification automatically

Aim of unsupervised learning:

It is to find the clusters of similar i/ps in the data without being explicitly told that these datapoints belong to one class and those to a different class.

The algorithm has to discover the similarities for itself.

The supervised learning algorithms have aimed to minimise the error criteria (ie, sum-of-squares error) based on the difference between the targets and the o/ps.

To find/compute the error, target data should be available, which is not true for unsupervised learning.

So, we require something else for learning.

- we need to find something internal to the algorithm.
  ⇒ means the measure has to be independent of the task as we can't change the algorithm.
- In supervised learning, error criteria is task-specific as it was based on the target data that is specified.

ER-SRIT

⇒ If two i/ps are close together, then it means their vectors are similar and so the distance between them is small (Euclidean distance).

⇒ The i/ps which are close together are identified being similar so that they can be clustered, while i/ps that are far away are not clustered together.

This can be extended to the nodes of a network by aligning weight space with the input space.

- If the weight values of a node are similar to the elements of an input vector then the node should be a good match for the i/p and any other i/ps that are similar.

This idea can be seen in a simple clustering algorithm, the k-Means algorithm.

K-Means Algorithm:

Ex:- Group of tourists (data) with a couple of tourguides (centroids) clusters.

- Suppose that we want to divide our input data into k categories, where we know the k value.
- we allocate k cluster centers to the input space and position the centers so that there is one cluster center in the middle of each cluster.
- However, we don't know where the clusters are, and where their middle is, so we need an algorithm to find them.
- To find middle of a set of points, we require two things:
  * A distance measure ie., Euclidean distance
  * A mean average, once the distance is measured then we can compute the central point of a set of data pts using mean average.

k-Means Algorithm:

* **Initialization:**
  - choose a value for k.
  - choose k random positions in the i/p space.
  - assign the cluster centers μ_j to those positions.

* **Learning:**
  - repeat
    * for each datapoint x_i:
      - compute the distance to each cluster center
      - assign the datapoint to the nearest cluster center with distance

$$d_i = \min_j d(x_i, \mu_j)$$

    * for each cluster centre:
      - move the position of the centre to the mean of the points in that cluster.

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^{N_j} x_i$$

where N_j is the no. of pts in the cluster j

  - until the cluster centres stop moving.

- Usage
  →

ER-SBIT

* Usage :

- for each test point :
  * compute the distance to each cluster centre.
  * assign the datapoint to the nearest cluster center with distance.

$$d_i = \min_j d(x_i, \mu_j)$$

⇒ We start by positioning the cluster centres randomly through the input space, as we don't know where to put them,

⇒ Then we update their positions according to the data.

⇒ We decide which cluster each datapoint belongs to by computing the distance between each datapoint and all of the cluster centres, and assigning it to the cluster that is closest.

⇒ This computational cost is reduced by using KD-Tree algo.

⇒ for all of the points that are assigned to a cluster, we then compute the mean of them, and move the cluster center to that point(place).

⇒ Iterate the algorithm until the cluster centres stop moving.

- The algorithm is susceptible to local minima: depending upon where the centres are initially positioned, we get very different solutions.
- There are cases we don't know in advance how many clusters are there in the data.
- By running the algorithm many different times, and we use many diff centre locations, and the solution that minimises the overall 'sum-of-squares error is likely to be the best one.

→ If we set k to equal to the no. of datapoints, we can position one centre on every datapoint and the sum-of-squares error is zero, resulting in overfitting.

Dealing with noise:

* The mean average, which is the central to the algo, is very susceptible to outlier, ie, very noisy measurements.
  - one way to avoid is replace the mean with median, which is known as robust statistic, that is not affected by outliers.

Problems:
- Problems with the noise data
- choosing the no. of clusters (k value)

EB-SBIT

Ex:- {2, 4, 10, 12, 3, 20, 30, 11, 25}

Initial cluster centroids: $m_1 = 4$, $m_2 = 11$

Data	Distance to $m_1 = 4$	Distance to $m_2 = 11$	Cluster	Distance to $m_3 = 3$	Distance to $m_4 = 18$	Cluster	m_5	m_6	
2	$(2-m_1)^2 = 2$	9	c_1	1	16	c_1	2.75	17.6	c_1
4	0	7	c_1	1	14	c_1	0.75	15.6	c_1
10	6	1	c_2	7	8	c_2	5.25	5.6	c_2
12	8	1	c_2	9	6	c_2	7.25	7.6	c_2
3	1	8	c_1	0	15	c_1	1.75	16.6	c_1
20	16	9	c_2	17	2	c_2	15.25	0.4	c_2
30	26	19	c_2	27	12	c_2	25.25	10.4	c_2
11	7	0	c_2	8	7	c_2	6.25	8.6	c_2
25	21	14	c_2	22	7	c_2	20.25	5.4	c_2

$c_1 = \{2, 4, 3\}$,

$c_2 = \{10, 12, 20, 30, 11, 25\}$.

$d = \sqrt{(x_2 - x_1)^2}$,

$m_3 = 3$ $\quad m_4 = 18$.

$= \frac{2+4+3}{3} = 3$.

$c_1 = \{2, 4, 10, 15\}$ $\quad m_5 = 4.75$

$c_2 = \{12, 20, 30, 11, 25\}$ $\quad m_6 = 19.6$.

$c_1 = \{2, 4, 10, 4, 12, 3\}$ $\quad m_7 = 7$

$c_2 = \{20, 30, 25\}$. $\quad m_8 = 25$